

Segurança da Informação

Relatório Técnico Final — AP01

Análise e Melhoria de Segurança em Sistema Web Acadêmico

Versão final — entrega 12/05/2026

Disciplina	Segurança da Informação
Aluno	Vinicius Steuernagel
Turma	5º Semestre — Engenharia de Software — Católica SC
Professor	Edson Vaz Lopes
Data	12 de maio de 2026
Repositório	github.com/[usuario]/seguranca-da-informacao-AP01
Sistema publicado	[usuario].github.io/seguranca-da-informacao-AP01

1. Descrição Resumida do Sistema

O sistema analisado é um protótipo web para registro de ocorrências acadêmicas, desenvolvido com HTML5, CSS3 e JavaScript puro, sem back-end real, API ou banco de dados centralizado. Toda a lógica de negócio, autenticação e persistência ocorrem exclusivamente no navegador, via *localStorage* e *sessionStorage*.

Funcionalidades presentes na versão original recebida:

- Login com usuários fixos no código-fonte (senhas em texto plano)
- Cadastro de ocorrências com dados pessoais: nome, matrícula, CPF, e-mail e telefone
- Listagem e busca sem filtro por perfil — todos viam tudo
- Alteração de status e exclusão de registros sem restrição de perfil
- Exportação total em JSON incluindo senhas em texto plano
- Logs de auditoria apagáveis por qualquer usuário
- Troca livre de perfil (ALUNO → ADMIN) sem verificação

A versão entregue neste trabalho passou por análise crítica e implementação de melhorias de segurança compatíveis com o escopo front-end, detalhadas nas seções seguintes.

2. Regras de Negócio Percebidas

2.1 Autenticação

O login compara e-mail e hash SHA-256 da senha com a lista de usuários. A sessão é representada por um JWT simulado (HMAC-SHA256) com expiração de 30 minutos, armazenado no *localStorage*. Tentativas inválidas são limitadas a 5 por sessão de navegador.

2.2 Perfis e Controle de Acesso

Três perfis com permissões distintas, definidas em um objeto *PERMISSIONS* centralizado. O perfil é fixado no token JWT no momento do login — não há seletor de perfil na interface.

- ALUNO: visualiza apenas suas próprias ocorrências (filtro por matrícula do token). Sem ações administrativas.
- PROFESSOR: cria ocorrências e altera status. Não acessa CPF, obs. interna, logs nem exportação.
- ADMIN: acesso completo — todas as colunas, ações, logs e exportação.

2.3 Ocorrências

- Cada ocorrência tem: ID único (UUID), nome, matrícula, e-mail, categoria, prioridade, descrição, obs. interna, status, autor e data.
- CPF e telefone foram removidos do formulário (minimização de dados — LGPD).
- Todos os campos são sanitizados antes de persistir e escapados antes de renderizar.

2.4 Logs e Exportação

Logs registram: data/hora, usuário, perfil, ação e detalhe. Limpeza restrita ao ADMIN. Exportação disponível apenas ao ADMIN e não inclui hashes de senha.

3. Identificação dos Ativos

Item	Ativo identificado	Por que tem valor?
1	Dados pessoais dos alunos (nome, matrícula, e-mail)	Dados protegidos pela LGPD. Vazamento gera responsabilidade legal e danos ao titular.
2	Registros de ocorrências acadêmicas	Representam situações sensíveis. Alteração indevida afeta a vida escolar do aluno.
3	Credenciais de usuários (hashes SHA-256)	Permitem acesso ao sistema. Comprometimento afeta autenticidade de todos os registros.
4	Logs de auditoria	Base para rastreabilidade. Perda ou adulteração impede reconstrução de eventos.
5	Código-fonte da aplicação	Visível via DevTools. Contém regras de negócio e hashes — inspecionável por qualquer usuário.
6	Token JWT no localStorage	Representa a sessão ativa. Comprometimento permite impersonar o usuário.
7	Observações internas das ocorrências	Informações restritas à coordenação/admin. Não devem ser visíveis a alunos.

4. Classificação dos Dados e Ativos

Item	Dado ou ativo	Classificação	Justificativa
1	Senhas dos usuários	Restrito	Credencial de acesso. Armazenadas como hash SHA-256 — nunca em texto plano.
2	Observação interna	Confidencial	Uso interno da instituição. Visível apenas ao ADMIN.
3	Registros de ocorrências	Confidencial	Dados acadêmicos vinculados a alunos identificados. Acesso controlado por perfil.
4	Logs de auditoria	Interno	Restritos à administração. Necessários para rastreabilidade.
5	Nome e matrícula do aluno	Interno	Identificadores acadêmicos. Acessíveis apenas a perfis autorizados.
6	E-mail institucional	Interno	Dado de contato. Visível para PROFESSOR e ADMIN conforme necessidade.
7	Código-fonte	Público	Inspecionável via DevTools por qualquer usuário — limitação inerente ao front-end.

5. Análise dos Riscos Encontrados na Versão Original

Item	Problema identificado	Risco associado	Impacto possível
1	Senhas em texto plano no código-fonte	Comprometimento de credenciais	Qualquer pessoa com acesso ao código obtém todas as senhas.
2	Token fictício exposto (FAKE_API_TOKEN)	Falsa segurança	Em sistema real, token exposto permite acesso não autorizado à API.
3	Troca livre de perfil sem restrição	Escalada de privilégios	Aluno se tornava ADMIN e acessava todas as funcionalidades.
4	Exportação irrestrita com senhas em texto plano	Vazamento massivo de dados e credenciais	Qualquer usuário exportava CPF, senhas e observações internas de todos.
5	Logs apagáveis por qualquer usuário	Destruição de evidências	Ações maliciosas encobertas por eliminação de logs.
6	Aluno acessava dados de todos os alunos	Quebra de privacidade — LGPD	Exposição de dados pessoais de terceiros sem autorização.
7	innerHTML sem escape em dados do usuário	XSS — Cross-Site Scripting	Dado malicioso armazenado no localStorage executado como HTML/JS.
8	Math.random() para gerar IDs	Colisão de IDs de ocorrências	IDs duplicados causam sobrescrita silenciosa de registros.
9	Sessão sem expiração automática	Acesso indevido em sessão abandonada	Computador compartilhado mantém sessão ativa indefinidamente.
10	Sem limite de tentativas de login	Ataque de força bruta às credenciais	Tentativas ilimitadas de adivinhar senhas sem bloqueio.
11	Coleta de CPF e telefone sem necessidade	Violação da minimização de dados — LGPD Art. 6º, III	Dados desnecessários aumentam o impacto de qualquer vazamento.
12	Senhas de demo expostas na tela de login	Quebra de confidencialidade das credenciais	Qualquer pessoa que abra o sistema vê as senhas sem precisar inspecionar o código.
13	Sem Content-Security-Policy	Sem barreira contra injeção de scripts externos	Sem CSP, scripts de origens externas podem ser injetados e executados.

6. Melhorias Implementadas no Código

#	Melhoria implementada	Arquivo	O que mudou?
1	Hash SHA-256 nas senhas (Web Crypto API)	app.js	Senhas nunca em texto plano. Login compara hash da entrada com hash armazenado.
2	JWT com assinatura HMAC-SHA256	app.js	Token com header, payload e assinatura real. Verificado a cada ação sensível.
3	Expiração de sessão 30 min + contador na topbar	app.js	Token com campo 'exp'. Sessão encerrada automaticamente ao expirar.
4	RBAC real — troca de perfil removida	app.js / index.html	Perfil fixado no token. Seletor de perfil removido. Permissões verificadas por ação.
5	Aluno vê apenas suas próprias ocorrências	app.js	Filtro em render() por studentId extraído do token JWT.
6	PROFESSOR sem acesso a obs. interna, logs e exportação	app.js	Objeto PERMISSIONS centraliza regras. Colunas e botões ocultos por perfil.
7	Exportação restrita ao ADMIN, sem hashes	app.js	Verificação de permissão. Hashes de senha excluídos do payload exportado.
8	Logs imutáveis para não-admins	app.js	Limpeza de logs disponível apenas para ADMIN.
9	esc() em todos os pontos de innerHTML (anti-XSS)	app.js	Função esc() aplica escape completo (&, <, >, ", ', /) em 22 pontos de renderização.
10	crypto.randomUUID() para IDs de ocorrências	app.js	Substitui Math.random(). IDs criptograficamente únicos, sem risco de colisão.
11	Rate limiting simulado: bloqueio após 5 tentativas	app.js	Contador em sessionStorage. Bloqueia formulário após 5 falhas consecutivas.
12	Senhas removidas da tela de login	index.html	Senhas não mais exibidas na interface. Referência ao README para credenciais de teste.
13	Campo CPF removido do formulário	index.html	Minimização de dados (LGPD). CPF não é necessário para o fluxo de ocorrências.
14	Content-Security-Policy no HTML	index.html	Meta CSP: default-src 'self', bloqueia scripts e estilos de origens externas.
15	Aviso visível de protótipo didático	index.html	Banner fixo informa que o sistema não deve receber dados reais.

7. Tabela Principal: Problema — Risco — Controle — Implementado?

Item	Problema	Risco	Controle	Implementado?
1	Senhas em texto plano	Vazamento de credenciais	Hash SHA-256 via Web Crypto API	Sim
2	Token fictício exposto	Falsa segurança	JWT com assinatura HMAC-SHA256	Sim
3	Troca livre de perfil	Escalada de privilégios	Perfil no token; seletor removido	Sim
4	Exportação irrestrita com senhas	Vazamento massivo	Export só para ADMIN; senhas excluídas	Sim
5	Logs apagáveis por todos	Perda de rastreabilidade	Limpeza restrita ao ADMIN	Sim
6	Aluno vê dados de todos	Quebra de privacidade	Filtro por studentId no token	Sim
7	innerHTML sem escape (XSS)	Execução de código malicioso	esc() em todos os 22 pontos de renderização	Sim
8	Math.random() para IDs	Colisão de registros	crypto.randomUUID()	Sim
9	Sessão sem expiração	Sessão abandonada ativa	Token exp + encerramento automático	Sim
10	Sem limite de tentativas de login	Força bruta	Rate limiting: bloqueio após 5 falhas	Sim
11	Coleta excessiva (CPF, telefone)	Viola LGPD Art. 6º, III	Campos removidos do formulário	Sim
12	Senhas expostas na tela de login	Exposição de credenciais	Removidas — consultar README	Sim
13	Sem Content-Security-Policy	Injeção de scripts externos	Meta CSP adicionada no HTML	Sim
14	Autenticação real ausente	Login contornável via DevTools	Back-end com auth centralizada e BD	Dep. back-end
15	Autorização só no front-end	Regras ignoráveis via console	Middleware de autorização no servidor	Dep. back-end
16	Logs manipuláveis (localStorage)	Auditoria não confiável	Logs em banco de dados imutável	Dep. banco de dados
17	Dados sem criptografia em repouso	localStorage acessível	Criptografia no banco de dados	Dep. banco de dados

Item	Problema	Risco	Controle	Implementado?
18	Sem HTTPS	Dados interceptáveis em trânsito	TLS/SSL + HSTS no servidor	Dep. infraestrutura
19	Rate limiting real ausente	Força bruta real	Rate limiting no servidor por IP/conta	Dep. back-end
20	Sem backup centralizado	Perda irreversível de dados	BD com backup e retenção definida	Dep. banco de dados

8. Justificativas Técnicas

8.1 Hash SHA-256 e JWT com HMAC-SHA256

Problema identificado: O sistema original armazenava senhas em texto plano em `const USERS`, visível via DevTools, e usava um token fictício sem verificação. A sessão era um JSON puro no `localStorage`, facilmente forjável.

Controle implementado: Implementada a Web Crypto API nativa para: (1) gerar hash SHA-256 da senha no login e comparar com o hash armazenado; (2) gerar JWT com assinatura HMAC-SHA256 real, verificada a cada ação sensível. O token carrega `sub`, `role`, `studentId`, `iat` e `exp`.

Conceitos relacionados: Conceitos: confidencialidade, autenticidade, integridade, sessão segura.

Limitação residual: A chave de assinatura JWT está no código-fonte — inaceitável em produção. SHA-256 sem salt é vulnerável a rainbow tables. Validação real exige back-end.

8.2 RBAC Real e Filtro por Perfil

Problema identificado: Qualquer usuário alterava o próprio perfil para ADMIN via seletor. Alunos acessavam dados de todos os outros alunos. Obs. interna e logs eram visíveis a todos.

Controle implementado: Perfil fixado no token JWT no momento do login. Seletor removido. Objeto `PERMISSIONS` centraliza as permissões. Função `can(role, acao)` verifica antes de cada operação. Na função `render()`, `ALUNO` filtra por `studentId` do token.

Conceitos relacionados: Conceitos: autorização, RBAC, menor privilégio, privacidade, LGPD.

Limitação residual: Verificações ocorrem no front-end — contornáveis via console do navegador. Autorização real exige middleware no servidor validando cada requisição.

8.3 Proteção contra XSS (esc()) em innerHTML

Problema identificado: Todos os dados do `localStorage` eram inseridos via `innerHTML` sem escape. Um dado malicioso armazenado (ex.: `[img src=x onerror=alert(1)]`) seria executado como HTML no próximo render, comprometendo todos os usuários que visualizassem o registro.

Controle implementado: Criada a função `esc()` que escapa `&`, `<`, `>`, `"`, `'` e `/` antes de qualquer inserção via `innerHTML`. Aplicada em 22 pontos de renderização: tabela de ocorrências, logs e mensagens de status. Dados de controle (`role`, `category`, `priority`) também são escapados.

Conceitos relacionados: Conceitos: integridade, XSS, sanitização, defesa em profundidade.

Limitação residual: A sanitização no front-end é uma camada adicional, não substituta. A proteção definitiva contra XSS armazenado exige validação no servidor.

8.4 Rate Limiting Simulado e `crypto.randomUUID()`

Problema identificado: O formulário de login permitia tentativas ilimitadas — ataque de força bruta sem qualquer resistência. Os IDs de ocorrências usavam `Math.random()`, sujeito a colisões que sobrescrevem registros silenciosamente.

Controle implementado: Implementado contador de tentativas em `sessionStorage` (resetado ao fechar a aba). Após 5 falhas, o botão de login é desabilitado e o usuário é informado. IDs substituídos por `crypto.randomUUID()` — criptograficamente único, sem colisões.

Conceitos relacionados: Conceitos: disponibilidade, integridade, controle de acesso, criptografia.

Limitação residual: O rate limiting em `sessionStorage` é contornável recarregando a aba. Proteção real exige controle por IP ou por conta no servidor.

8.5 Minimização de Dados e Remoção de Senhas da Interface

Problema identificado: O formulário coletava CPF e telefone sem necessidade para o fluxo de ocorrências. As senhas de demonstração eram exibidas em texto puro na tela de login, visíveis a qualquer pessoa que abrisse o sistema.

Controle implementado: CPF e telefone removidos do formulário (LGPD Art. 6º, III). Senhas removidas da interface — a tela de login agora instrui consultar o README para credenciais de teste. Content-Security-Policy adicionada bloqueando scripts e estilos de origens externas.

Conceitos relacionados: Conceitos: minimização de dados, privacidade por design, LGPD, CSP.

Limitação residual: A CSP em meta tag tem limitações em relação à CSP via cabeçalho HTTP do servidor. A proteção completa exige configuração no servidor web.

9. Melhorias que Dependem de Back-end, BD ou Infraestrutura

#	Melhoria necessária	Depende de quê?	Por que não resolve só no front-end?
1	Autenticação real com hash+salt e sessão de servidor	Back-end + BD	Hashes no código-fonte são inspecionáveis. O servidor precisa verificar e emitir tokens que o cliente não possa forjar.
2	Autorização validada em cada requisição	Back-end	Regras no JS são contornáveis via console. A validação deve ocorrer no servidor antes de retornar qualquer dado.
3	Logs persistentes e imutáveis	Banco de dados	localStorage é modificável pelo usuário. Logs confiáveis exigem BD com controle de acesso e append-only.
4	Criptografia de dados em repouso	BD + infraestrutura	localStorage é texto puro acessível a qualquer script da página. Criptografia real ocorre no BD.
5	HTTPS / TLS	Infraestrutura	Dados em trânsito sem TLS são interceptáveis. Exige certificado e configuração no servidor web.
6	Rate limiting real por IP	Back-end	Contador em sessionStorage é contornável recarregando a página. Controle real é no servidor.
7	Backup e retenção de dados	BD + infraestrutura	localStorage pode ser apagado pelo usuário ou pelo navegador. BD com snapshot automatizado é necessário.
8	Adequação formal à LGPD	Governança	Exige DPO, AIPD, mapeamento de dados, consentimento, prazos de retenção e plano de resposta a incidentes.

10. Limitações da Solução Entregue

Mesmo com todas as melhorias implementadas, as seguintes limitações estruturais permanecem:

- O código-fonte é inspecionável via DevTools — hashes SHA-256 e a chave de assinatura JWT são visíveis.
- O JWT é verificado apenas no front-end — pode ser forjado via console do navegador.
- SHA-256 sem salt é vulnerável a rainbow tables e ataques de dicionário pré-computados.
- O rate limiting em sessionStorage é contornável recarregando a aba ou em aba anônima.
- A CSP via meta tag tem menor eficácia do que via cabeçalho HTTP do servidor.
- Os logs permanecem no localStorage — podem ser manipulados via DevTools pelo usuário.
- Não há HTTPS, proteção CSRF real, revogação de token nem controle de infraestrutura.
- A persistência é local: cada navegador tem sua própria base, sem backup centralizado.

11. Conclusão

O sistema, da forma como foi recebido originalmente, não poderia ser usado em produção com dados reais.

A versão original apresentava falhas críticas e acumuladas: senhas em texto plano visíveis no código-fonte; token fictício sem verificação; qualquer usuário podia se tornar administrador; alunos acessavam dados de todos os outros alunos; exportação irrestrita incluindo senhas; logs apagáveis sem restrição; dados renderizados via innerHTML sem escape (vulnerabilidade XSS); IDs gerados com Math.random() (colisões possíveis); sem limite de tentativas de login; e coleta desnecessária de CPF e telefone.

Esses problemas comprometiam simultaneamente a **confidencialidade** (dados expostos a perfis não autorizados), a **integridade** (registros manipuláveis, XSS, IDs duplicados), a **disponibilidade** (dados sem backup, logs destruíveis) e a **autenticidade** (sessão forjável, perfil alterável). Havia ainda exposição direta à LGPD.

A versão entregue implementou **15 melhorias efetivas** no código: hash SHA-256 nas senhas, JWT com HMAC-SHA256, expiração de sessão com contador, RBAC sem troca de perfil, filtro de ocorrências por matrícula, restrição de exportação e logs, escape anti-XSS em 22 pontos de renderização, IDs por UUID criptográfico, rate limiting simulado, remoção de CPF do formulário, remoção de senhas da interface, Content-Security-Policy, aviso de protótipo e sanitização de entradas.

O sistema melhorado representa um avanço substancial dentro das limitações do ambiente front-end. Entretanto, **mesmo a versão melhorada não está apta para uso em produção com dados reais**. As proteções implementadas são simulações didáticas — corretas em conceito, mas contornáveis por usuários com conhecimento técnico básico.

Para tornar o sistema realmente utilizável em produção, seriam necessários:

- Back-end com autenticação centralizada, hash com salt (bcrypt/Argon2) e sessão de servidor
- Banco de dados com controle de acesso, criptografia em repouso, logs imutáveis e backup
- Validação de autorização no servidor em cada requisição
- HTTPS com certificado TLS válido e cabeçalhos de segurança (CSP via header, HSTS)
- Rate limiting real por IP/conta no servidor
- Análise de Impacto à Proteção de Dados (AIPD) e adequação formal à LGPD
- Testes de segurança (SAST, DAST, pentest) antes de qualquer implantação

A segurança de uma aplicação não pode ser avaliada apenas pela aparência ou funcionamento visual. Uma análise técnica cuidadosa revela riscos profundos em sistemas construídos sem processo formal de segurança. Esse é o raciocínio que esta atividade propõe desenvolver — e que deve orientar toda decisão de Engenharia de Software com responsabilidade sobre dados de pessoas reais.